BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division:C Batch:2 Practical No:2

Roll no: 47 PRN No: 12412632 Name of Student: **Krishna Meghwani**

Subject Name : Deep Learning

Problem Statement :

Implement a Multi-Layer Perceptron (MLP) neural network to classify the Iris flower dataset into its three species (Setosa, Versicolor, and Virginica) based on the four input features (sepal length, sepal width, petal length, and petal width), and evaluate the model using accuracy, a confusion matrix, and a classification report.

Algorithm :

• Import the required libraries from scikit-learn (dataset, preprocessing, model, and metrics).

• Load the Iris dataset and separate it into feature matrix X and target vector y.

• Split the data into training (80%) and testing (20%) sets using train_test_split with a fixed random_state for reproducibility.

• Standardize the features using StandardScaler: fit on the training data and apply the same transform to the test data.

• Create an MLPClassifier with two hidden layers of 10 neurons each, ReLU activation, and the Adam optimizer.

• Train (fit) the MLP model on the standardized training data.

• Predict the class labels for the standardized test data.

• Evaluate the model by computing the accuracy score, confusion matrix, and classification report.

Code :

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

iris=load_iris()
X,y=iris.data,iris.target
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
mlp=MLPClassifier(hidden_layer_sizes=(10,10),activation='relu',solver='adam',max_iter=1000,random_state=42)
mlp.fit(X_train,y_train)
y_pred=mlp.predict(X_test)
print("Accuracy:",accuracy_score(y_test,y_pred))
print("Confusion Matrix:\n",confusion_matrix(y_test,y_pred))
print("\nClassification Report:\n",classification_report(y_test,y_pred))
```

✓ 2.3s

Output :

```
Accuracy: 0.9666666666666667
Confusion Matrix:
 [[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      0.89      0.94         9
           2       0.92      1.00      0.96        11

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30
```